BIG DATA ANALYTICS · COURSE PROJECT REPORT

Large-Scale Social Media Sentiment and Trend Analytics Using MongoDB

2.94 million real tweets, MongoDB aggregation pipelines, NLP, Apache Spark, a Parquet data lake and a sharded cluster

Submitted by

Sahaj Saliya	23BIT128
Hil Kalathiya	23BIT092

Department of Information and Communication Technology

October 2026

Live website: https://www.info.sahaj.si/BDA-Analytics/
Source code: https://github.com/SahajIVVIX-1/BDA-Analytics

Contents

1 Abstract 3

2 Introduction 3

3 Problem Statement 3

4 Objectives 4

5 Existing System 4

6 Proposed System 5

7 Big Data Characteristics 5

7.1 The five Vs 5

7.2 CAP theorem 6

7.3 Mapping to the Hadoop ecosystem 7

8 Architecture 8

9 Technology Stack 9

10 Dataset 10

11 MongoDB Design 11

12 Data Preprocessing 13

13 Sentiment Analysis 14

14 Topic Analysis 16

15 Trend Detection 16

16 Engagement Analytics 17

17 Anomaly Detection 17

18 API 18

19 React Dashboard 18

20 Big Data Lab: Spark, Data Lake and Sharding 18

20.1 How Spark processes the data 19

20.2 The data lake: bronze, silver and gold 20

20.3 Gold insights 22

20.4 Sharding MongoDB 23

21 Performance Experiments 24

22 Results 25

22.1 Big Data Lab results 29

23 Limitations 30

24 Future Scope 31

25 Conclusion 31

26 References 32

1 Abstract

Social media platforms produce large volumes of short, noisy, multilingual text whose patterns cannot be found by reading individual posts. This project builds a complete batch analytics platform for such data on a single 16 GB machine, with MongoDB as both the storage layer and the main computation engine. 2,944,814 real tweets from the FiveThirtyEight / Clemson University dataset of Internet Research Agency accounts are streamed, validated, cleaned and bulk-loaded into a schema-validated MongoDB collection with twelve purpose-built indexes. Lightweight CPU-based NLP adds sentiment (TF-IDF with logistic regression, test macro-F1 0.58 on TweetEval), topics (NMF, 16 English and 12 Russian topics) and keywords. MongoDB aggregation pipelines materialise daily, hourly, hashtag, keyword and account rollups, which a FastAPI service combines with live aggregation through a simple query planner. A React dashboard presents sentiment, topics, trends, engagement, geography, anomalies and a searchable data explorer. Experiments at 100K, 500K, 1M and 2.95M documents measure ingestion throughput, batch size and parallelism effects, index build strategy, indexed versus collection-scan queries, aggregation scaling, the benefit of pre-aggregation, and NLP throughput. Engagement counts are absent from the source and are generated synthetically, and are labelled as such throughout. A Big Data Lab extension processes the same archive with Apache Spark into a bronze / silver / gold Parquet data lake (147 s for all rows, 2.48x faster on 4 cores than on 1, and up to 20x faster queries than CSV through partition pruning) and loads it into a three-shard MongoDB cluster to compare hashed and ranged shard keys.

2 Introduction

Platforms such as Twitter / X carry public conversation at a scale where manual reading is impossible. Analysts want to know how sentiment changes, which topics and hashtags grow, which days are unusual and which accounts drive activity. Answering these questions needs three things: storage that copes with semi-structured documents, processing that does not require all data to fit in memory, and analytics that can be explored interactively.

This project implements those three layers with MongoDB, Python and React, and measures how they behave as data grows. The core platform stays on one machine to show which Big Data techniques (streaming, batching, server-side aggregation, indexing, pre-aggregation, parallelism) already matter at the scale of a few million records. A second part, the Big Data Lab, then processes the same data with the distributed tools of the course: Apache Spark, a partitioned Parquet data lake and a sharded MongoDB cluster.

3 Problem Statement

Given millions of social media posts with timestamps, text and author metadata:

1. ingest them reliably from common file formats without exhausting memory, rejecting invalid and duplicate records and reporting what happened;
2. store them so that analytical queries by time, sentiment, topic, language, location and hashtag are fast;
3. derive sentiment, topics, trends and anomalies;
4. present the results through an API and an interactive dashboard;
5. measure how the system performs as the data grows, on hardware a student has.

4 Objectives

- Build a batch ingestion pipeline for CSV, JSON and JSONL with validation, cleaning, de-duplication, bulk insertion and statistics.
- Design a MongoDB schema, validation rules and indexes justified by the query patterns.
- Implement sentiment analysis, topic extraction, keyword extraction, trend detection, engagement analytics, time analysis and anomaly detection, computing aggregates inside MongoDB.
- Expose the analytics through a documented REST API with validation and pagination.
- Build a professional React dashboard with filtering, search and clear handling of loading, error and empty states.
- Run and record performance experiments at several data sizes.
- Document limitations honestly, including which data is synthetic.

5 Existing System

Typical approaches available to an analyst without a dedicated platform:

- **Native platform analytics** (for example account dashboards) show engagement for one's own account only and give no access to cross-account text analysis.
- **Spreadsheets and scripts** (Excel, pandas loading a CSV) work for thousands of rows but need the whole file in memory, have no indexes and must recompute every statistic from scratch.
- **Relational databases** can store posts, but hashtags, mentions and nested metadata require several joined tables and schema changes for new sources.
- **Cluster frameworks** (Hadoop, Spark) handle very large data but need a cluster or substantial setup, and on their own do not give an analyst a dashboard. This project uses Spark in its Big Data Lab and measures where it helps and where its start-up cost dominates.
- **Commercial social listening tools** are capable but closed, paid, and do not expose their methods for academic evaluation.

The gap: an open, reproducible, single-machine system that handles millions of posts, keeps the processing close to the data, and documents and measures its own methods.

6 Proposed System

A layered batch analytics platform (see section 8):

- streaming readers and a multi-process cleaning pipeline that bulk-loads MongoDB;
- a document schema with server-side validation and indexes chosen per query pattern;
- NLP enrichment written back to the documents;
- aggregation pipelines that materialise rollup collections and refresh only affected days after new uploads;
- a FastAPI service with a query planner that answers from rollups when possible and from live aggregation when needed;
- a React dashboard;
- an experiment harness that records every measurement with its hardware;
- a **Big Data Lab extension** (`bigdata/`, `docs/bigdata.md`): an Apache Spark pipeline that builds a bronze / silver / gold Parquet data lake from the raw files, and a sharded MongoDB cluster (config server, three shards, `mongos`) that compares hashed and ranged shard keys. The core platform runs without it.

7 Big Data Characteristics

7.1 The five Vs

Characteristic	In this project	How it is handled	Honest qualification
Volume	2,944,519 posts from 13 CSV files (1.14 GB); 3.5 GB uncompressed BSON in MongoDB plus 1.0 GB of indexes; 4.4M rows in the keyword rollup	Spark processes all rows in 147 s on 4 cores; Parquet stores silver in 498 MB; 1M posts spread over 3 shards	large for a laptop, small for industry
Velocity	new data arrives through batch ingestion jobs; rollups refresh incrementally	batch ETL at up to 26,185 rows/s (Spark), incremental `$merge`	no real-time stream; velocity is simulated by uploads
Variety	free text in 56 language values, emoji, URLs, nested author metadata, arrays of entities, CSV / JSON / JSONL inputs, optional real engagement in uploads	schema-on-read bronze, typed silver schema, MongoDB documents with `$jsonSchema` and multikey indexes	
Veracity	duplicate ids, empty posts, missing language and	validation with rejection reasons, de-duplication	handled by validation, flags and documented

Characteristic	In this project	How it is handled	Honest qualification
	location, spam-like posts, a collection gap, timestamps without time zone	(1,392 ids), lineage columns, quality flags, synthetic data labelled	assumptions
Value	trends, anomalies and sentiment patterns not visible from individual posts	gold tables and rollups answer dashboard questions in milliseconds; partitioned Parquet answers ad-hoc questions 8-20× faster than CSV	findings describe a troll-account archive, not the public

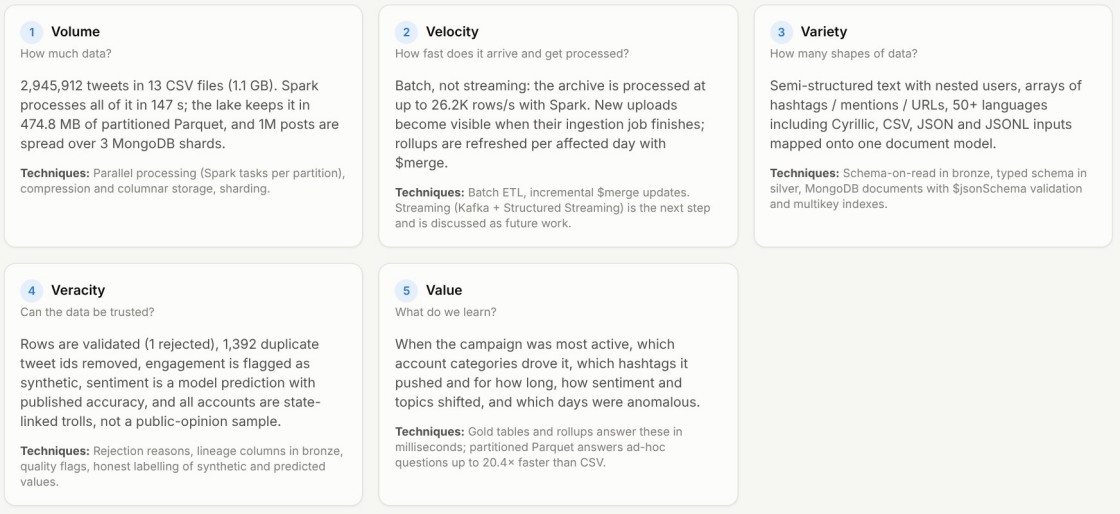


Figure: the five Vs with measured numbers (Big Data Lab, Course concepts tab)

7.2 CAP theorem

In a distributed database, a network partition forces a choice between consistency (every read sees the latest write) and availability (every request gets an answer). MongoDB is **CP by default**: each shard is a replica set with a single primary that accepts writes, so two nodes never accept conflicting writes; if the primary is cut off, that shard refuses writes until a new primary is elected (seconds of unavailability). The trade-off is tunable per operation: `readPreference: secondaryPreferred` serves possibly stale reads for more availability, and `writeConcern: {w: 1}` acknowledges faster than the default `majority` at the risk of a rollback after failover. In a sharded cluster an unreachable shard only affects queries that need its chunks, which is a second reason, besides speed, to choose a shard key that matches the main queries. The experiment cluster uses one member per replica set, so it demonstrates routing and distribution but not fault tolerance. The Parquet lake gets consistency the HDFS way: files are written once and replaced as a whole.

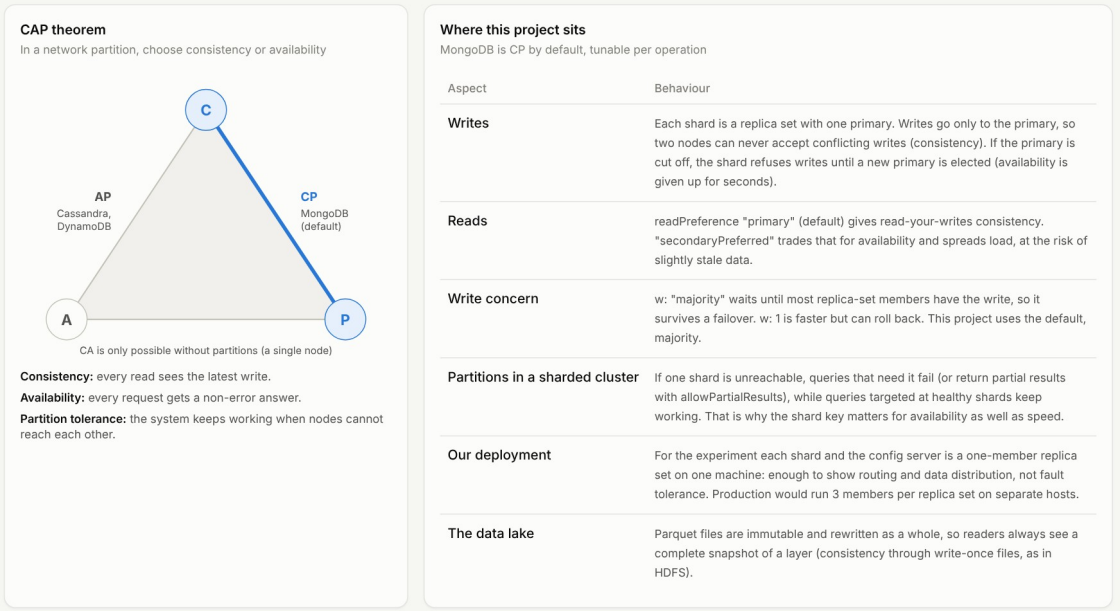


Figure: CAP theorem and where MongoDB sits

7.3 Mapping to the Hadoop ecosystem

Concept	Classic tool	In this project
Distributed storage	HDFS	Parquet data lake partitioned by year/month (`data/lake`)
Batch processing	MapReduce	Spark stages (partial aggregate → shuffle → final aggregate); MongoDB `$group`
SQL on files	Hive	Spark SQL over Parquet with partition and column pruning
NoSQL store	HBase / Cassandra	MongoDB documents with 12 indexes
Horizontal scaling	region servers	MongoDB sharding with hashed and ranged shard keys
OLAP cubes	Kylin, materialised views	gold tables and MongoDB rollup cubes
Resource manager	YARN	Spark's local scheduler (`local[N]`); standalone, YARN or Kubernetes on a cluster
Streaming	Kafka + Spark Streaming	not implemented (future work)

8 Architecture

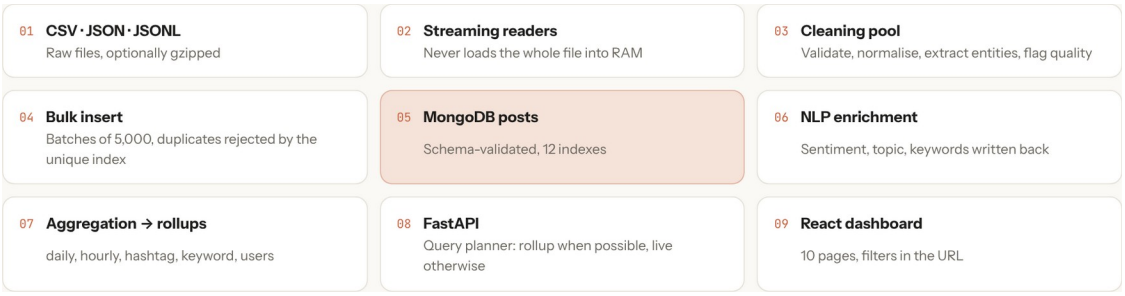


Figure: core platform data flow, from raw files to the React dashboard

The Big Data Lab extension adds a second, distributed path from the same raw files:

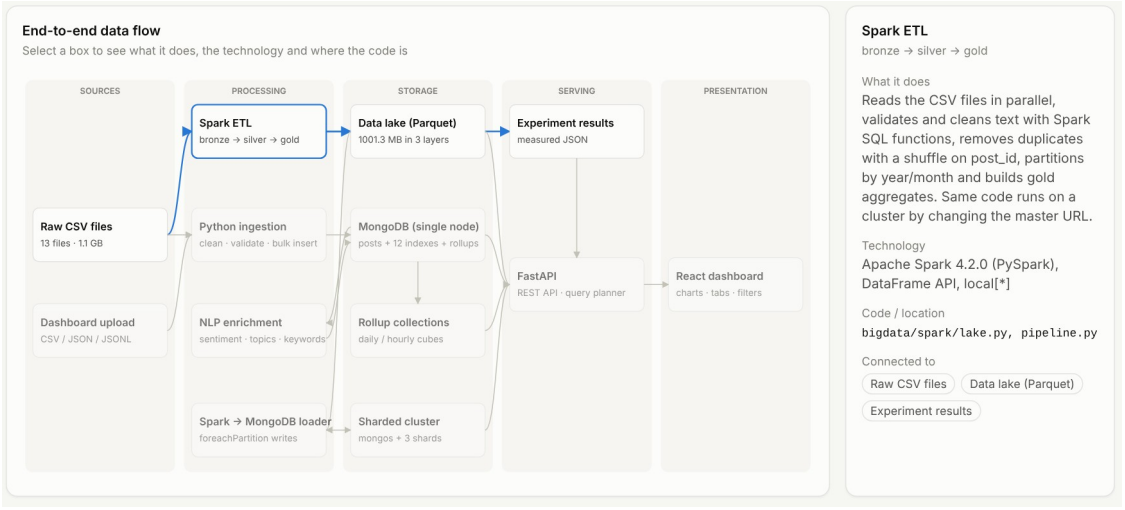


Figure: Big Data Lab data flow (Spark ETL, Parquet lake, sharded cluster, dashboard)

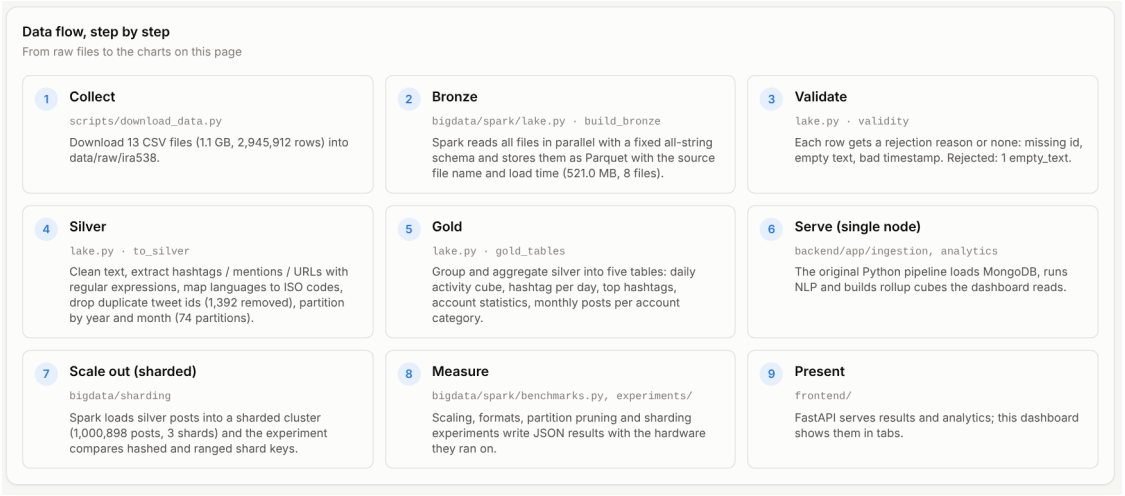


Figure: the Big Data Lab step by step, with the code location of each step

Components and design decisions are described in `docs/architecture.md` and `docs/bigdata.md`. Key decisions: MongoDB is the system of record and the aggregation engine; dashboard queries read pre-aggregated rollups when the filters allow; processing is batch-oriented and parallel within one machine.

9 Technology Stack

Layer	Technology (version used)	Why
Database	MongoDB 8.0.23 Community, WiredTiger	document model, multikey indexes, aggregation framework
Driver	PyMongo 4.18	official driver, bulk operations, aggregation
Backend	Python 3.13, FastAPI 0.142, Pydantic 2.13, Uvicorn	typed validation, automatic OpenAPI docs
NLP / ML	scikit-learn 1.9 (TF-IDF, logistic regression, Naive Bayes, NMF, Isolation Forest), vaderSentiment 3.3.2, NumPy 2.5	CPU-only, fast, well understood
Reporting	pandas 3.0 (experiment report generation)	tabulating measured results
Frontend	React 19, Vite 8, Tailwind CSS 4, Recharts 3, React Router 7, lucide-react	modern component UI, fast builds, declarative charts
Testing	pytest 9, FastAPI TestClient, Vitest 5, Testing Library, ruff, oxlint	
Distributed processing (extension)	Apache Spark 4.2 (PySpark), Java 21	parallel ETL over partitions; the same code runs on a cluster
Lake storage (extension)	Apache Parquet (snappy), Hive-style partitions	columnar, compressed, partition and column pruning
Scale-out database (extension)	MongoDB sharded cluster: config server RS, 3 shard RS, `mongos`	horizontal scaling by shard key
Containers (extension)	Docker Compose	the Spark image and cluster need no local Java or MongoDB install

Technology stack

What each Big Data component is and why it was chosen

Layer	Technology	Why
Distributed processing	Apache Spark 4 (PySpark)	Parallel ETL over partitions; same code scales from a laptop to a cluster
Storage format	Apache Parquet + snappy	Columnar, compressed, splittable; supports partition pruning and column pruning
Data lake layout	Medallion: bronze / silver / gold	Raw data is kept, cleaning is repeatable, aggregates are cheap to serve
Operational database	MongoDB 8	Flexible documents, multikey indexes on arrays, aggregation framework
Horizontal scaling	MongoDB sharding (config RS, 3 shard RS, mongos)	Splits a collection across machines by a shard key
Batch NLP	scikit-learn	CPU-only sentiment and topic models
API	FastAPI + Pydantic	Typed REST endpoints, OpenAPI docs at /docs
Dashboard	React 19 + Vite + Tailwind + Recharts	Lazy-loaded pages, URL state, dark mode
Containers	Docker Compose	One command starts Spark and the sharded cluster with no local Java or MongoDB install

Figure: technology used at each stage of the Big Data Lab

Not used: Streamlit, GPUs, transformer models. Spark is used only in the Big Data Lab extension (`bigdata/`); the core ingestion, NLP and dashboard pipeline does not depend on it.

10 Dataset

See `docs/dataset.md` for full details.

- **Source:** FiveThirtyEight `russian-troll-tweets`, collected by Linvill and Warren (Clemson University), CC BY 4.0. Tweets from accounts Twitter linked to the Internet Research Agency.
- **Size:** 2,946,207 rows read, 2,944,814 stored, 2,843 accounts, 2012-02-02 to 2018-05-30 (99.5% in 2015-2017).
- **Fields used:** text, publish date, language, account region, author, followers / following, account category and type, post type.
- **Missing:** sentiment labels (predicted by a model trained on TweetEval), engagement counts (generated synthetically and flagged), time zone (UTC assumed), city-level location.
- **Representativeness:** the accounts are a state-linked influence operation; results do not describe public opinion.

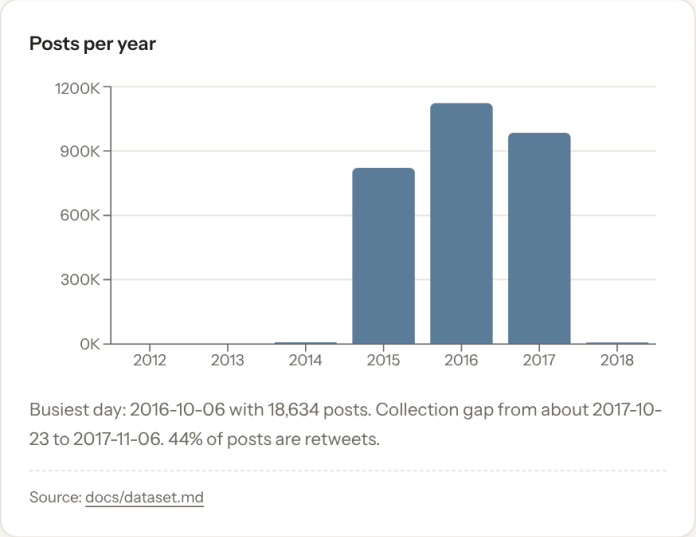


Figure: posts per year in the dataset

Fields and how they are stored

SOURCE COLUMN	STORED AS	NOTES
`tweet_id`	`post_id`	Unique index; duplicates rejected
`content`	`text, clean_text`	Cleaned text drives NLP and search
`publish_date`	`created_at`	No time zone in the source; UTC assumed
`language`	`language`	Language names mapped to ISO 639-1 codes
`region`	`location.country`	Account-level country; no city
`author`	`user.username`	—
`alt_external_id`	`user.user_id`	external_author_id lost precision in the CSV
`followers, following, updates`	`user.*`	Follower count also stored as reach
`account_category, account_type`	`user.*`	Researcher-coded themes (RightTroll, LeftTroll, NewsFeed, ...)
`post_type, retweet`	`post_type, is_retweet`	—
`(none)`	`hashtags, mentions, url_count, emojis`	Extracted from the text during cleaning

Sentiment was trained and evaluated on a separate dataset, TweetEval (train 45,615 / validation 2,000 / test 12,284), which is never stored in MongoDB. Raw data is downloaded by `scripts/download_data.py` and is not in the repository.

Source: docs/dataset.md

Figure: source fields and how they are stored

11 MongoDB Design

See `docs/database.md`.

- **Collections:** `posts` (2.94M), five rollups (`daily_cube` 130K, `hourly_cube` 171K, `hashtag_daily` 418K, `keyword_daily` 4.4M, `users` 2.8K), `topics`, `ingestion_jobs`, `performance_tests`.
- **Document model:** user, location, engagement, sentiment and topic embedded as sub-documents; hashtags, mentions and keywords as arrays; original and cleaned text both kept.

- **Validation:** `$jsonSchema` with required fields and BSON types, `validationAction: error`.
- **Indexes:** unique `post_id`; date; compound (sentiment, date), (language, date), (country, date), (hashtags, date), (topic, date), (user, date desc); text-hash; partial index on unprocessed posts; sparse engagement index; text index. Compound indexes follow the equality-sort-range rule.
- **Aggregation:** `$match`, `$group`, `$project`, `$sort`, `$limit`, `$unwind`, `$lookup`, `$facet`, `$setWindowFields`, `$dateTrunc`, `$merge`, `$out`.



Figure: a real document from the posts collection

The 12 indexes on posts, and why each exists

NAME	KEYS	OPTIONS	WHY
`uniq_post_id`	`post_id`	unique	One document per source post; bulk inserts reject duplicates server-side (error 11000)
`created_at`	`created_at`	—	Date ranges and chronological sorting
`sentiment_created`	`sentiment.label, created_at`	—	Equality on sentiment + range on date
`language_created`	`language, created_at`	—	Equality on language + range on date
`country_created`	`location.country, created_at`	—	Geography filter + date range
`hashtags_created`	`hashtags, created_at`	multikey	Posts containing a hashtag, by date
`topic_created`	`topic.id, created_at`	—	Topic drill-down
`user_created`	`user.username, created_at desc`	—	One account's timeline, newest first
`text_hash`	`text_hash`	—	Find identical texts across accounts (copy-paste campaigns)
`unprocessed_partial`	`processed`	partial: processed=false	NLP job finds its next batch without scanning processed posts
`engagement_total`	`engagement.total desc`	sparse	Top-N posts by engagement without an in-memory sort
`text_search`	`clean_text (text)`	default_language: none	Keyword search; no stemming because the corpus is multilingual

Compound indexes follow the **equality, sort, range** rule, so one index serves "negative posts in October 2016, newest first". Low-cardinality fields such as `is_retweet` are deliberately not indexed.

Source: backend/app/database/indexes.py

Figure: the 12 indexes on posts and the query each one serves

12 Data Preprocessing

Steps (details in `docs/methodology.md`): streaming read; validation of id, text and timestamp; normalisation of language names to ISO codes and of unknown locations to null; HTML unescape and Unicode normalisation; extraction of hashtags, mentions, URLs and emoji; removal of retweet prefixes, URLs, mentions, emoji and control characters; quality flags; text fingerprint; batch bulk insert with server-side duplicate rejection.

Results of the full ingestion run (from the `ingestion_jobs` record):

Statistic	Value
Records read	2,946,207
Inserted	2,944,814
Duplicates (same `tweet_id`)	1,392
Invalid (empty text)	1
Hashtags extracted (distinct per post, summed)	1,753,395
Mentions extracted (distinct per post, summed)	1,153,675

Statistic	Value
URLs removed	2,827,842
Emoji removed (distinct per post, summed)	93,108
Posts without language	8,320
Posts without location	570,812
Flagged: link only / mention stuffing / short / hashtag stuffing / no text left / repetitive	123,874 / 104,415 / 96,748 / 39,176 / 32,125 / 1,068
Time (4 worker processes, batch 5,000)	316.3 s, 9,314 records/s

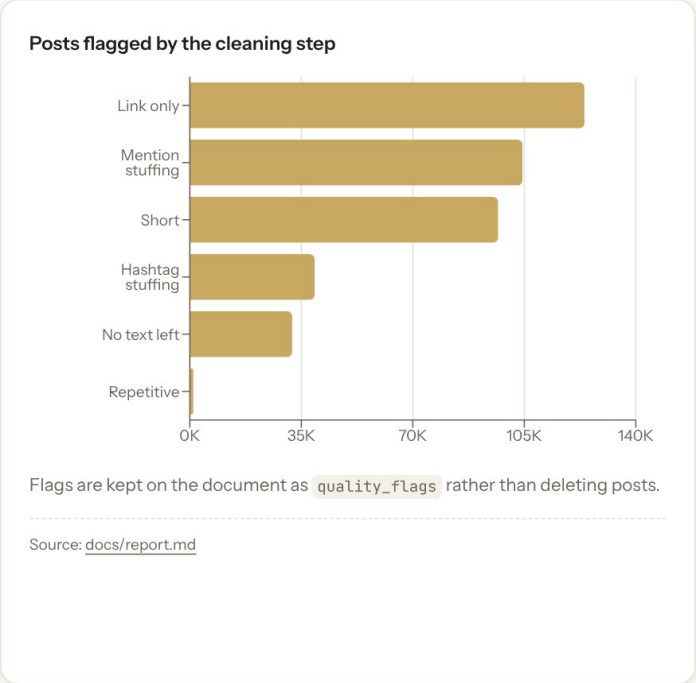


Figure: posts flagged by the cleaning step

13 Sentiment Analysis

Three-class sentiment for English posts. Three approaches were compared on the official TweetEval splits: VADER (lexicon), TF-IDF + Multinomial Naive Bayes, TF-IDF + logistic regression with the regularisation strength chosen on the validation split. Selection by validation macro-F1 picked logistic regression.

Measured on the TweetEval test split (12,284 tweets), from `experiments/results/sentiment_evaluation.json`:

Model	Test macro-F1	Test accuracy	Test macro recall
VADER	0.526	0.528	0.564

Model	Test macro-F1	Test accuracy	Test macro recall
TF-IDF + Naive Bayes	0.455	0.539	0.502
TF-IDF + logistic regression	**0.582**	**0.586**	**0.593**

For context, the TweetEval repository reports 62.9 macro recall for its SVM and FastText baselines and around 73 for transformer models; this project's model is below them. Applied to the 2,111,943 English posts it predicts 16.8% positive, 52.0% neutral and 31.2% negative. These are model outputs on out-of-domain text; their accuracy on this dataset has not been measured.

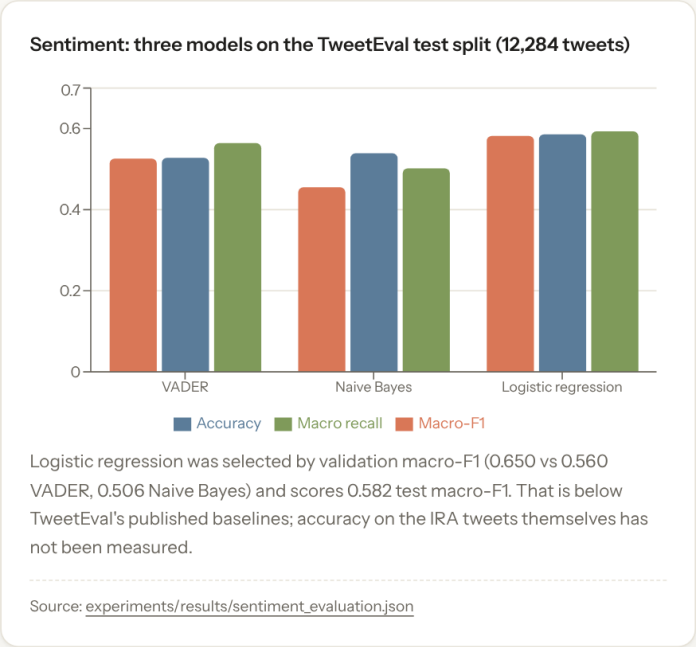


Figure: three sentiment models on the TweetEval test split

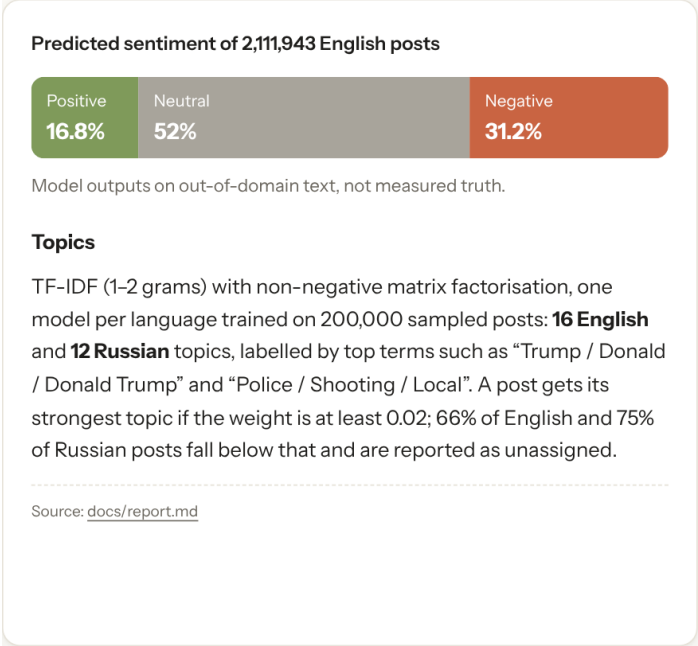


Figure: predicted sentiment of 2,111,943 English posts

14 Topic Analysis

TF-IDF (1-2 grams) and non-negative matrix factorisation, one model per language, trained on 200,000 sampled posts each: 16 English and 12 Russian topics, labelled by their top terms (for example “Trump / Donald / Donald Trump”, “Police / Shooting / Local”, “Black / Blacklivesmatter / Women”, “сша / трамп / обама”). A post is assigned its strongest topic if the weight is at least 0.02; 66% of English and 75% of Russian posts fall below that threshold and are reported as unassigned. Topic frequency, share, timeline, sentiment mix, engagement and 30-day growth are computed from the daily cube. The number of topics was chosen by inspection, not by a coherence metric.

15 Trend Detection

Window-over-window comparison over the daily hashtag, keyword and topic rollups, with `trend score = log10(1 + now) x log2((now + 5) / (before + 5)) x (1 + 0.1 x log10(1 + avg reach))` growth % and share growth %. A warning is raised when total volume changes more than threefold between the windows. Example: for the week ending 2016-11-08, `#trumpforpresident` rose from 1 to 1,682 posts (score 34.8), followed by `#sometimesitsokto`, `#2016electionin3words`, `#electionday` and `#hillaryforprison2016`.

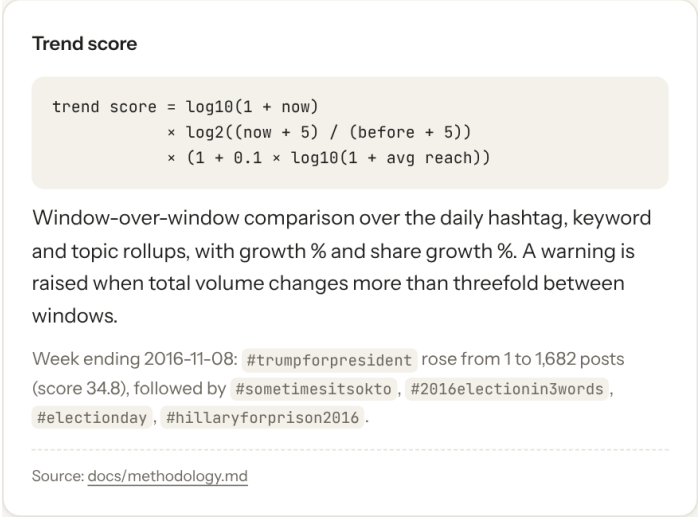


Figure: the trend score

16 Engagement Analytics

Totals of likes, comments and shares, engagement per 1,000 followers, engagement over time, by sentiment, by topic, and top posts (via a sparse descending index). **All engagement values in this dataset are synthetic**, generated from follower counts only (see `docs/methodology.md`), so the module demonstrates the pipeline and the queries, not real audience behaviour. Uploaded datasets with real engagement columns are analysed with the same code and flagged `synthetic: false`.

17 Anomaly Detection

Three methods on the daily series: a rolling 28-day z-score computed in MongoDB with `$setWindowFields` (volume and negative share, minimum 100 posts), the IQR rule, and an Isolation Forest on four daily features. On the full dataset with $|z| >= 3$, 128 of 1,699 days are flagged by at least one method. The largest volume spike, 2016-10-06 (18,634 posts against an expected 3,483, $z = 7.1$), is flagged by both the z-score and IQR rules; 2015-11-15 is flagged by three methods (volume spike, negativity surge, IQR).

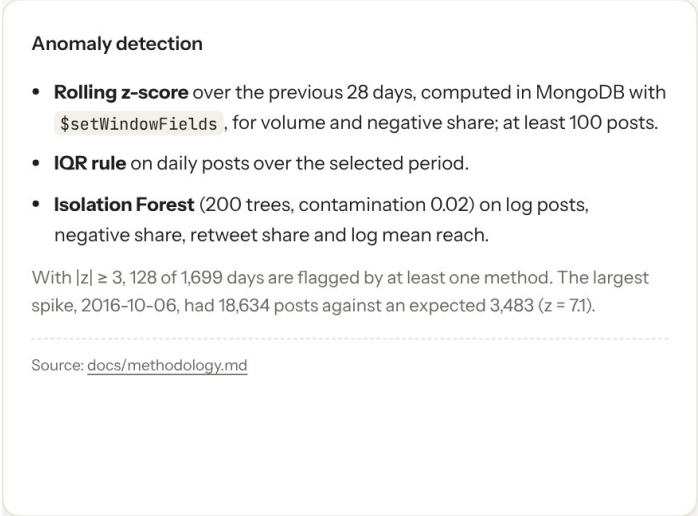


Figure: anomaly detection methods

18 API

26 FastAPI endpoints (`docs/api.md`): health, filter metadata, overview, time series, activity heatmap, sentiment, topics, trends, hashtags, engagement, geography, languages, anomalies, accounts, paginated posts, full-text search, post detail, upload, start ingestion, job list and detail, file preview, and four performance endpoints including live `explain()`. Shared Pydantic filter validation (422 on bad input), MongoDB error handlers (503 / 500 without connection details), pagination limits, gzip compression and a 120-second response cache.

19 React Dashboard

Eleven pages (including About dataset) plus the Big Data Lab page described in the next section. Pages: Overview, Sentiment, Trends, Topics, Engagement, Geography & language, Anomalies, Data explorer, Performance, Ingestion. Global filters (date presets, language, country, sentiment, topic, hashtag) are stored in the URL. Every chart shows which collection answered and how long it took, has loading skeletons, error states with retry, and empty states. Synthetic engagement is badged on every view. Light and dark themes, a collapsible sidebar and filter bar on phones, lazily loaded pages. Charts follow a colour-blind-checked palette with fixed category order.

20 Big Data Lab: Spark, Data Lake and Sharding

The Big Data Lab is the part of the project that applies the distributed tools of the course to the same 2.95M-post archive. It has three pieces, each measured, and a dashboard page with six tabs (Architecture, Spark processing, Data lake, Gold insights, Sharding, Course concepts) so the results are explored without scrolling through one long page.

20.1 How Spark processes the data

Spark reads the 13 CSV files into a DataFrame split into **partitions**. Transformations (`where`, `select`, `regexp_extract_all`, `groupBy`) are **lazy**: Spark builds a plan and only runs it when an action (`write`, `count`) needs a result. The plan is cut into **stages** at every **shuffle**; for example `dropDuplicates(post_id)` moves rows with the same id to the same task. Each stage runs one **task per partition**, one per core at a time. This is the MapReduce pattern: a partial aggregate on every partition (map side), an exchange of rows by key (shuffle), then the final aggregate (reduce side). The same code runs on a cluster by changing the master URL from `local[*]` to a standalone, YARN or Kubernetes master.

The pipeline has four stages:

1. **Bronze:** read every CSV file with a fixed all-string schema, add lineage columns (source file, load time) and write Parquet. Nothing is changed, so later steps can always be re-run from bronze.
2. **Validate:** every row gets a rejection reason or none (missing id, empty text, invalid timestamp).
3. **Silver:** clean the text with the same regular expressions and order as the Python pipeline, extract hashtags, mentions and URLs, map languages to ISO codes, drop 1,392 duplicate ids, and write Parquet partitioned by `year` and `month` (74 folders).
4. **Gold:** five small aggregate tables (daily activity, hashtag per day, top hashtags, account statistics, monthly category totals) that answer dashboard questions directly.

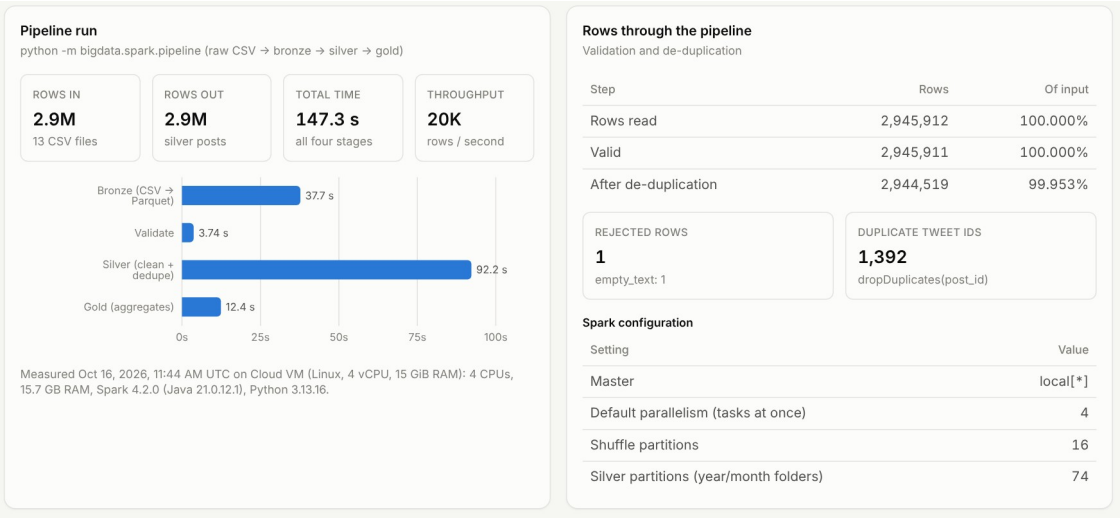


Figure: Spark pipeline run on all 2,945,912 raw rows

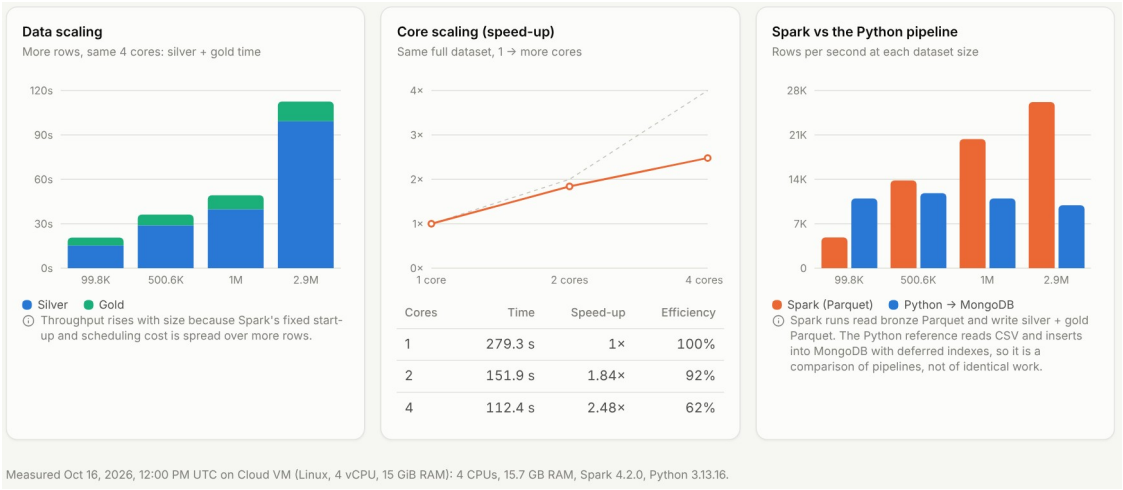


Figure: Spark data scaling and core scaling



Figure: physical plan of a one-month query; partial_count, Exchange hashpartitioning, count is the MapReduce pattern

20.2 The data lake: bronze, silver and gold

A data lake keeps data as files in open formats, organised in layers. **Parquet** is columnar: a query reads only the columns it uses (**column pruning**), and similar values in a column compress well. Partitioning by `year/month` creates one folder per month, so a filter on the month skips every other folder (**partition pruning**). Files are written once and replaced as a whole, which gives readers a consistent snapshot, the same model HDFS uses.

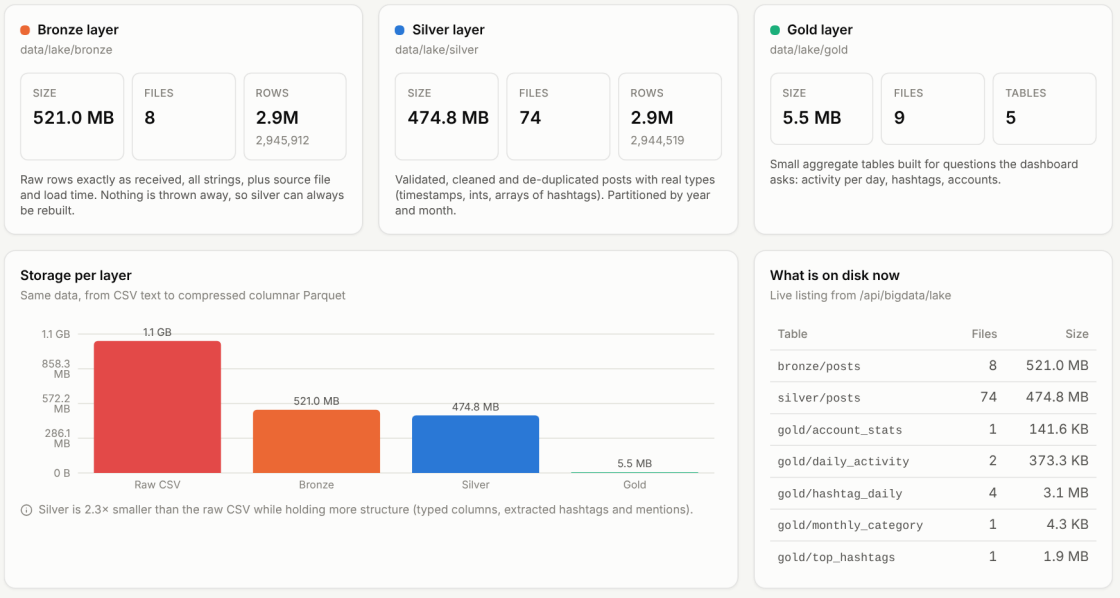


Figure: size and file count of each lake layer

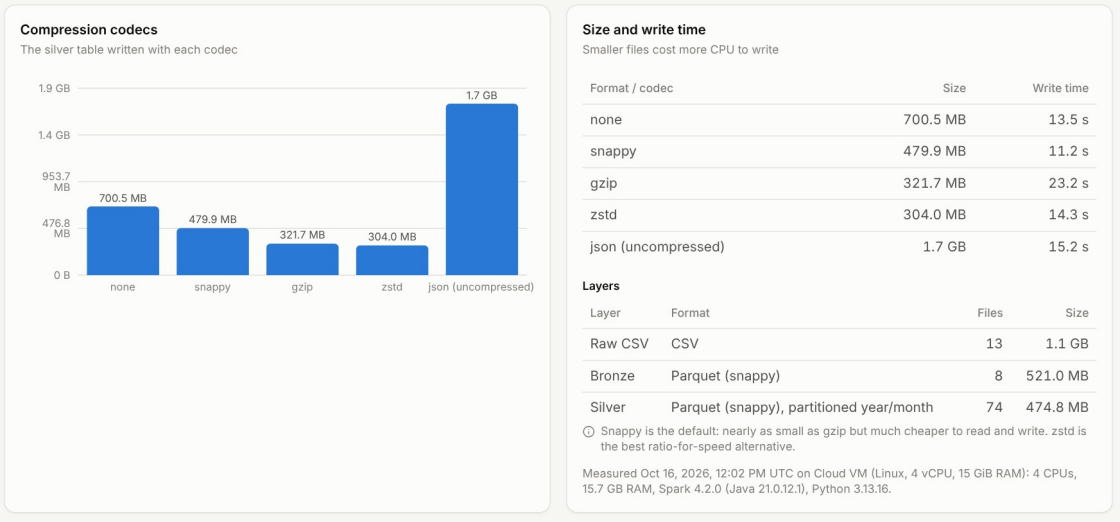


Figure: the silver table stored as JSON and as Parquet with four codecs

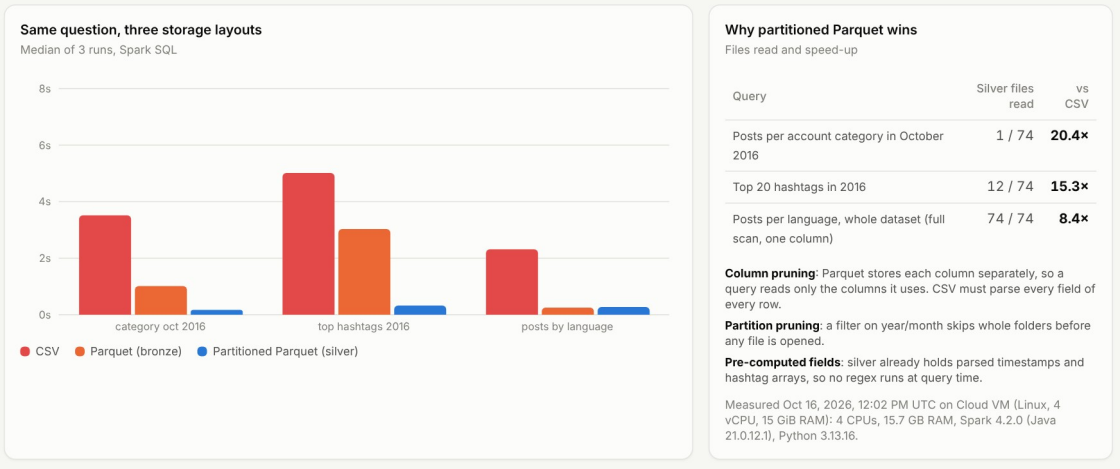


Figure: the same three questions on CSV, Parquet and partitioned Parquet

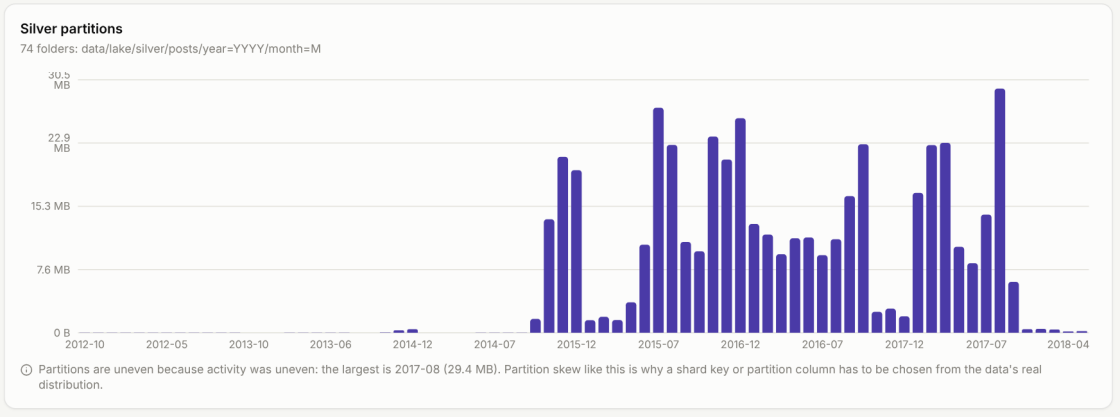


Figure: silver partitions by month; uneven because activity was uneven

20.3 Gold insights

The gold tables turn the cleaned posts into answers: activity per account category over time, the most used hashtags and how long each stayed active, the most active accounts, and the language and country mix.

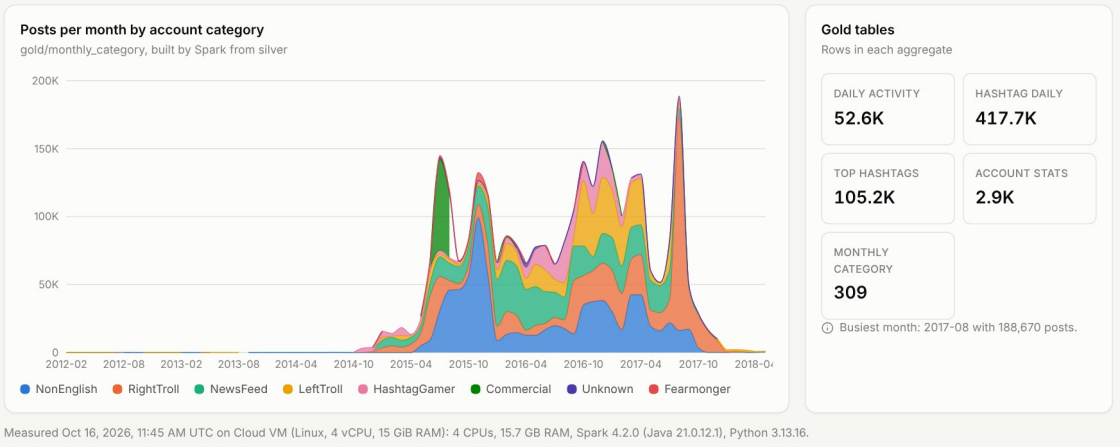


Figure: posts per month by account category, from the gold layer

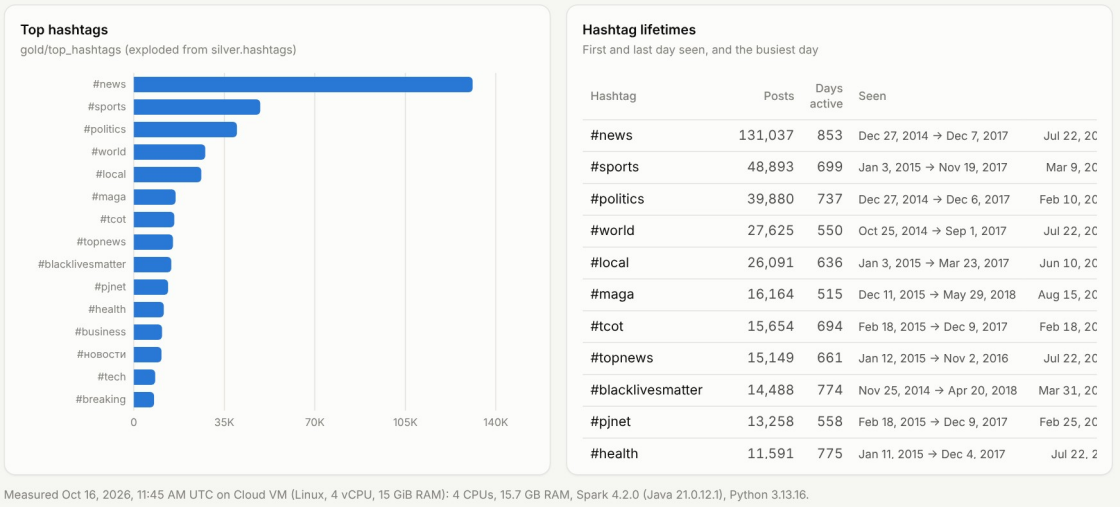


Figure: top hashtags from the gold layer

20.4 Sharding MongoDB

Sharding splits one collection across several servers (**shards**) by a **shard key**. The key's value range is cut into **chunks**; a **config server** stores which shard owns each chunk, and the **mongos** router uses that map to send each query only where it is needed. A query that filters on the shard key is **targeted** to one shard; any other query is **scatter-gather**: sent to all shards and merged. Spark loads the silver posts into the cluster in parallel with `foreachPartition`, each task inserting its partition with unordered bulk writes.

Two shard keys were compared on 1,000,898 posts:

- **hashed `post_id`:** spreads documents evenly and targets lookups by id, but every date-range query goes to all shards;
- **ranged `created_at`, pre-split at the terciles:** keeps each week on one shard, so date ranges are targeted, but lookups by id go to all shards. A monotonic key without pre-splitting would send every new insert to the last chunk (a hot shard).

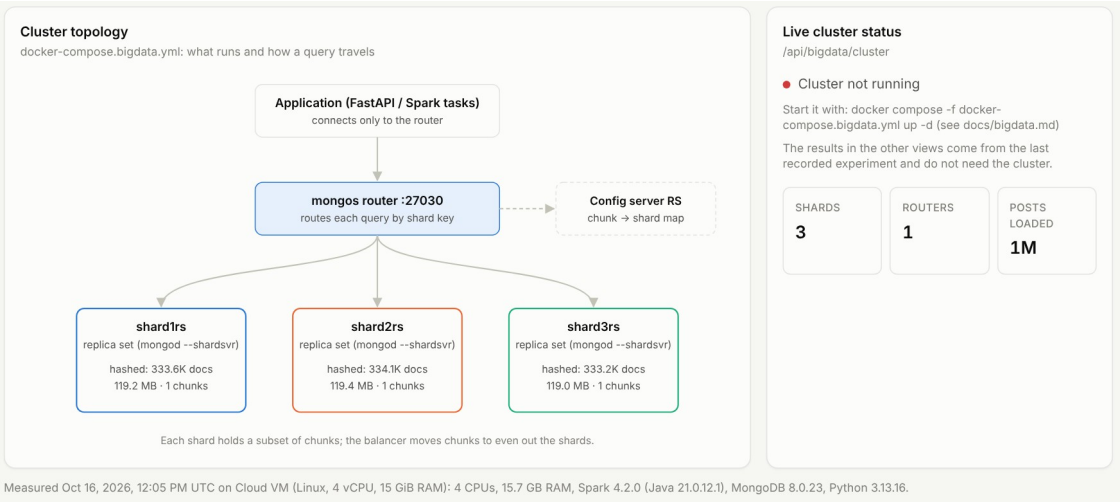


Figure: cluster topology: application, mongos router, config server and three shards

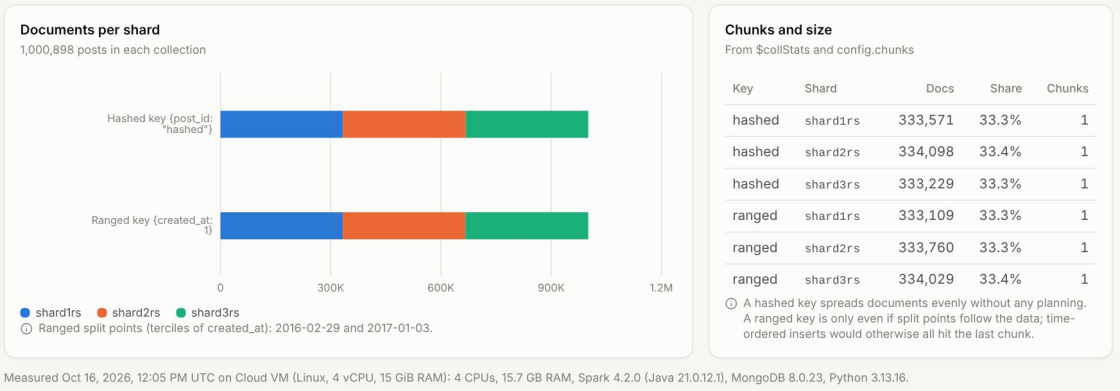


Figure: documents per shard for the hashed and ranged keys

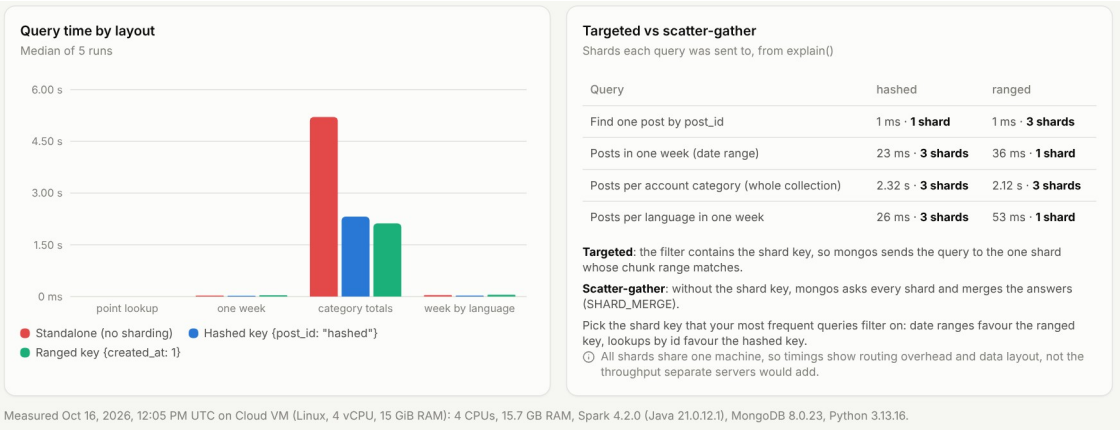


Figure: query time and number of shards targeted for each layout

21 Performance Experiments

Set-up, method and complete tables: `docs/performance.md` and `docs/methodology.md`.

Experiments:

1. ingestion scaling at 100K, 500K, 1M and 2.95M posts;
2. batch size (100 to 20,000);
3. number of cleaning worker processes (1, 2, 4);
4. building secondary indexes during or after the load;
5. six representative queries with and without their index, using `explain("executionStats")`;
6. six aggregation pipelines at each size, rollup versus live answers for five dashboard queries, and the full rollup rebuild;
7. NLP model throughput and the end-to-end enrichment job with 1, 2 and 4 workers.

The 5M size was not run: the real dataset has 2.95M posts and was not padded with synthetic data.

Big Data Lab experiments (`docs/bigdata.md`):

8. the Spark lake pipeline (bronze, silver, gold) on all 2.95M rows;

9. Spark data scaling (100K to 2.95M rows) and core scaling (1, 2, 4 cores);
10. storage formats and codecs (JSON, Parquet with none / snappy / gzip / zstd);
11. the same three questions on CSV, Parquet and partitioned Parquet;
12. a 3-shard MongoDB cluster with hashed and ranged shard keys against a standalone server (1M posts): load rate, distribution and query routing.

22 Results

Numbers in this chapter were measured on one cloud virtual machine (Intel Xeon @ 2.80 GHz, 4 vCPUs, 15.7 GB RAM, MongoDB 8.0.23 with a 4 GB WiredTiger cache, Python 3.13) on 2026-10-08. Full tables, parameters and run notes: `docs/performance.md`. Timings are medians of three runs unless stated.

Ingestion scales linearly. Throughput stayed between 9,916 and 11,815 records/s from 100K to 2.95M posts, so load time grew in proportion to data size: 9.1 s for 100K, 91 s for 1M, 297 s for 2.95M, plus 311 s to build the secondary indexes on the full collection (608 s in total).

Posts	Load (s)	Records/s	Index build (s)
100,000	9.1	10,992	9.5
500,000	42.3	11,815	46.9
1,000,000	91.0	10,992	99.1
2,944,814	297.1	9,917	310.9

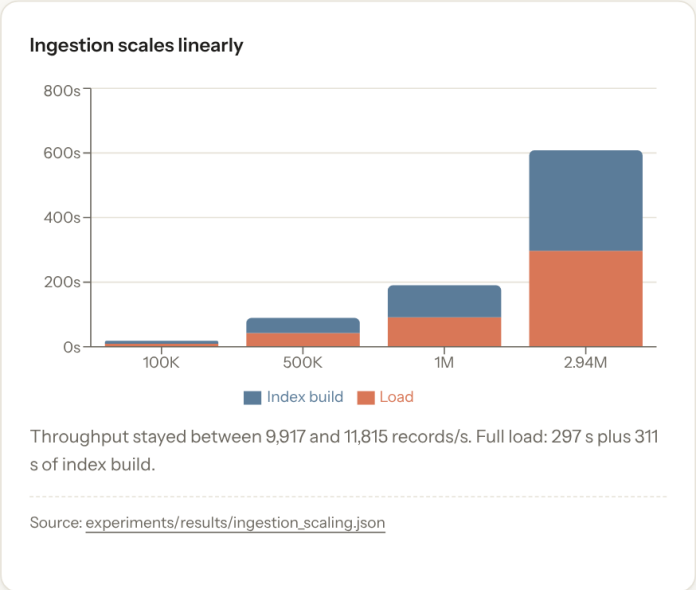


Figure: ingestion time by data size

Batch size. On 300,000 records, batches of 100 documents gave 7,909 records/s; 500 to 20,000 gave 11,184 to 11,795 records/s. Very small batches pay a round trip per batch; beyond about 500 the gain is small.

Parallel cleaning. One worker process gave 6,963 records/s, two gave 13,284 (1.9x), four gave 11,526. With 4 vCPUs shared by the workers, the inserting parent process and `mongod`, two workers were the best setting on this machine.

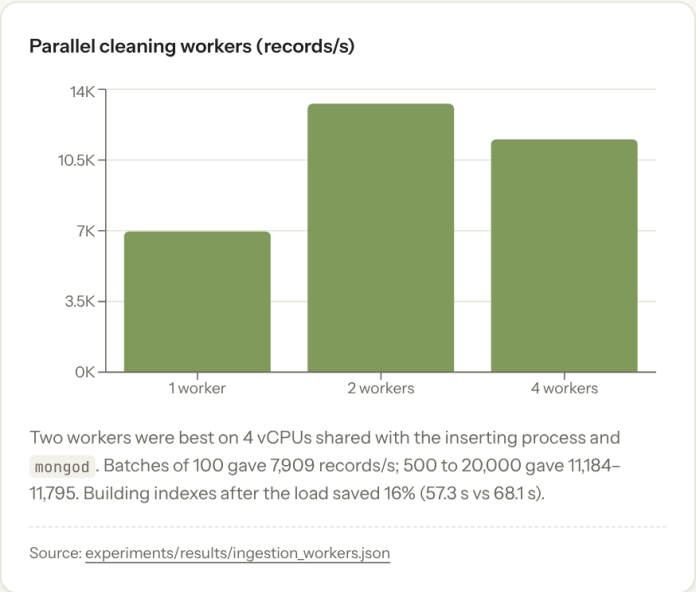


Figure: parallel cleaning workers

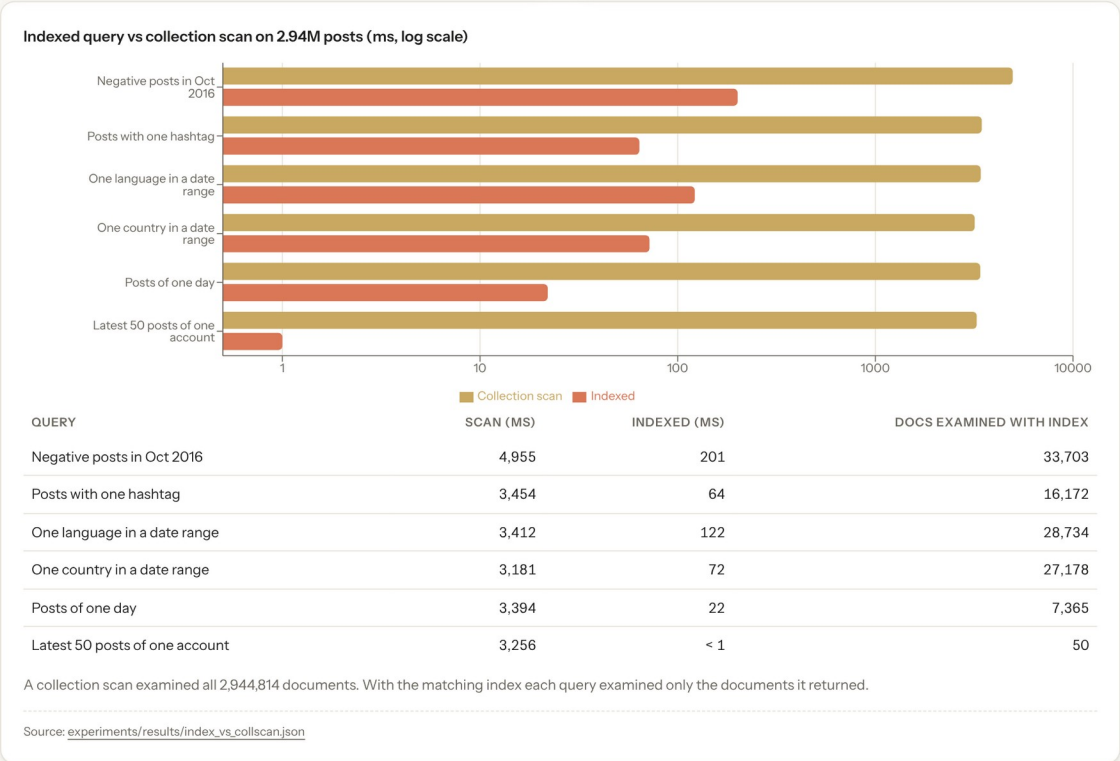
Index build timing. Loading 300,000 records with all 12 secondary indexes in place took 68.1 s; loading with only the unique index and building the rest afterwards took 27.9 s + 29.3 s = 57.3 s (16% less in total).

Indexes versus collection scans (2.94M posts). A collection scan examined all 2,944,814 documents and took 3.2 to 5.0 s. With the matching index each query examined only the documents it returned:

Query	Scan (ms)	Indexed (ms)	Documents examined with index
negative posts in October 2016	4,955	201	33,703
posts with one hashtag	3,454	64	16,172
one language in a date range	3,412	122	28,734
one country in a date range	3,181	72	27,178
posts of one day	3,394	22	7,365

Query	Scan (ms)	Indexed (ms)	Documents examined with index
latest 50 posts of one account	3,256	< 1	50

Scan time grew with collection size (for example 518 ms at 500K, 1,404 ms at 1M and 4,955 ms at 2.94M for the first query) while indexed time followed the number of matching documents.



QUERY	SCAN (MS)	INDEXED (MS)	DOCS EXAMINED WITH INDEX
Negative posts in Oct 2016	4,955	201	33,703
Posts with one hashtag	3,454	64	16,172
One language in a date range	3,412	122	28,734
One country in a date range	3,181	72	27,178
Posts of one day	3,394	22	7,365
Latest 50 posts of one account	3,256	< 1	50

A collection scan examined all 2,944,814 documents. With the matching index each query examined only the documents it returned.

Source: experiments/results/index_vs_collscan.json

Figure: indexed query versus collection scan on 2.94M posts (log scale)

Aggregation pipelines grow linearly with data. Each full-collection pipeline took about 0.2 to 0.9 s on 100K posts and 6.2 to 27.1 s on 2.94M posts (for example the sentiment distribution: 0.77 s, 3.19 s, 7.22 s, 21.26 s at 100K, 500K, 1M, 2.94M).

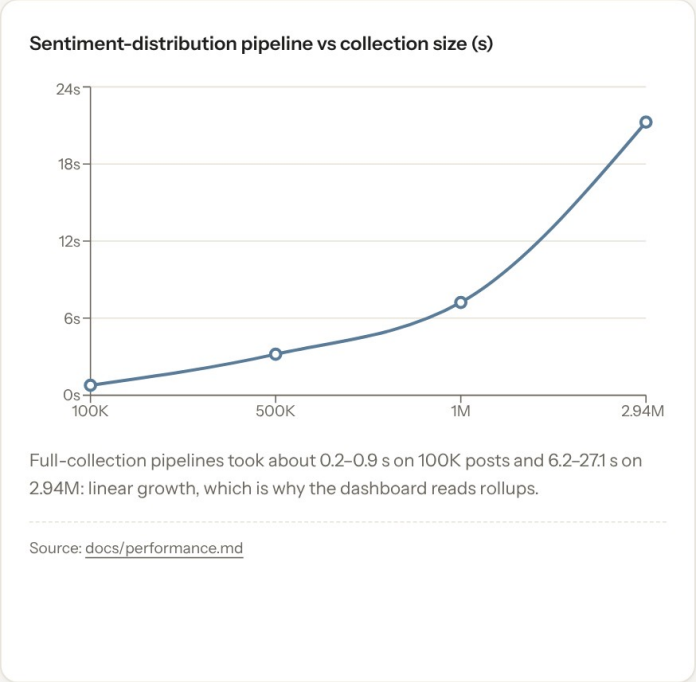


Figure: sentiment-distribution pipeline time versus collection size

Pre-aggregation gives the largest gain for the dashboard. The same queries answered from the `daily_cube` rollup instead of `posts`:

Dashboard query	Live on posts (s)	From rollup (s)	Speed-up
Overview KPIs	33.7	0.96	35x
Sentiment page	31.5	0.87	36x
Topics page	36.7	0.83	44x
Geography page	25.6	0.69	37x
Overview, English 2016	14.4	0.42	34x

The price is the rollup build: 470 s for a full rebuild of all five rollups, of which 333 s is the 4.4M-row keyword rollup. After an upload only the affected days are recomputed.

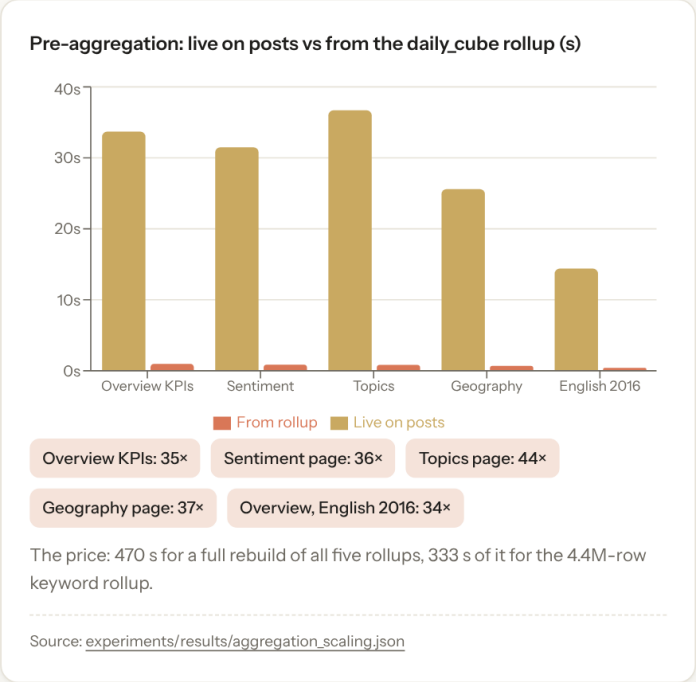


Figure: live aggregation versus the daily_cube rollup

NLP throughput. On 20,000 English posts: logistic regression sentiment 24,067 texts/s, VADER 17,767 texts/s, NMF topics plus keywords 25,677 texts/s. The end-to-end enrichment job (read, both models, bulk write) processed 200,000 posts at 5,213 posts/s with one worker and 10,355 posts/s with two; with two or more workers about 16 s of the 19 s were spent waiting on database writes, so writes, not models, became the limit. At the two-worker rate, enriching all 2.94M posts would take about 5 minutes (extrapolated, not measured as one run).

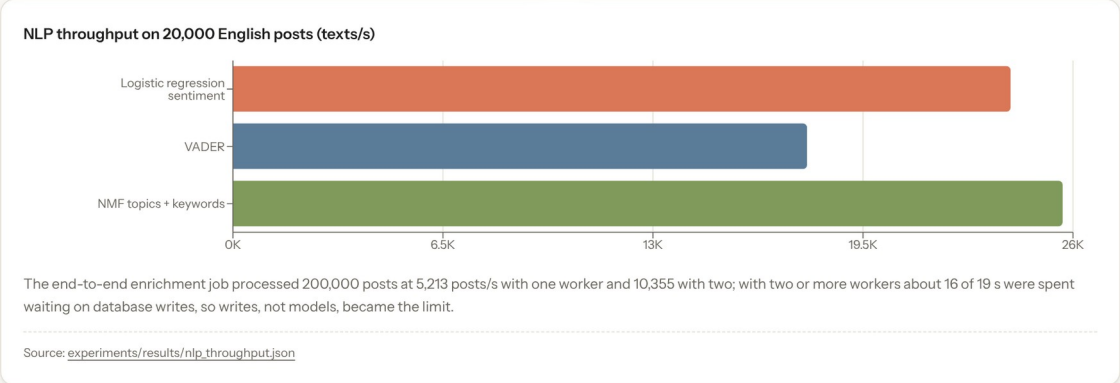


Figure: NLP throughput

Sentiment model. Validation macro-F1: logistic regression 0.650, VADER 0.560, Naive Bayes 0.506. Test macro-F1 0.582, accuracy 0.586, macro recall 0.593.

22.1 Big Data Lab results

Measured on a 4-vCPU Linux VM with Spark 4.2.0 and MongoDB 8.0.23 (details in `docs/bigdata.md`).

Spark pipeline. 2,945,912 raw rows became 2,944,519 silver posts in 147 s (bronze 37.7 s, validation 3.7 s, silver 92.2 s, gold 12.4 s), about 20,000 rows/s. The cleaned output matched the Python pipeline field for field on the 1,000-row sample.

Spark scaling. At 4 cores, 100K rows took 20.6 s and 2.94M rows 112.4 s (4,840 vs 26,185 rows/s): fixed start-up cost dominates small inputs. Going from 1 to 2 to 4 cores gave speed-ups of 1.84× and 2.48× (efficiency 92% and 62%).

Storage. Raw CSV 1.14 GB, bronze Parquet 546 MB, silver 498 MB in 74 partitions, gold 5.8 MB. The silver table was 1,834 MB as JSON, 735 MB as uncompressed Parquet, 503 MB with snappy, 337 MB with gzip and 319 MB with zstd.

Query layout. A one-month query took 3.51 s on CSV, 1.01 s on Parquet and 0.17 s on partitioned Parquet (20×; 1 of 74 folders read); top hashtags for 2016 took 5.02 s, 3.03 s and 0.33 s (15×).

Sharding. With 1M posts, the hashed key put 33.3-33.4% of documents on each shard; the ranged key, pre-split at the terciles, 33.3-33.4% too. Lookups by `post_id` went to 1 shard with the hashed key and to all 3 with the ranged key; one-week date ranges went to 1 shard with the ranged key and to all 3 with the hashed key. A whole-collection `$group` took 5.21 s standalone and 2.12-2.32 s sharded, because each shard aggregated a third in parallel. Writes through `mongos` were slower with the hashed key (51,160 vs 64,670 docs/s standalone).

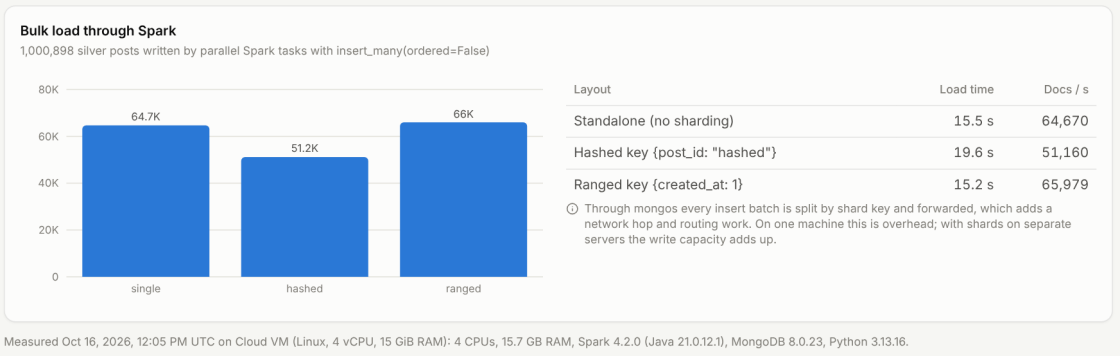


Figure: load rate through mongos

23 Limitations

- **Single machine.** Spark runs in local mode and all shards of the experiment cluster share one machine, with one member per replica set: the extension demonstrates distributed processing, routing and data distribution, not the throughput or fault tolerance of separate servers.
- **Batch, not real time.** New data becomes visible after its ingestion job and rollup refresh.
- **Dataset.** Troll-account archive, not representative; time zone assumed; account-level location only.
- **Synthetic engagement.** No real likes, replies or shares exist in the source.

- **Sentiment.** English only; test macro-F1 0.58 on TweetEval, below published baselines; not validated on this dataset; no sarcasm or context handling.
- **Topics.** Two thirds of posts unassigned at the chosen threshold; topic count chosen by inspection.
- **Experiments.** One cloud virtual machine with 4 vCPUs; warm cache only; 5M size not run; each timing is a median of three runs, so small differences are within noise.
- **Security.** No authentication on the API: intended for local use.

24 Future Scope

- Transformer sentiment models (for example a Twitter-specific RoBERTa) on a GPU, evaluated on a labelled sample of the target data.
- Topic count chosen by coherence; BERTopic or embedding-based clustering.
- Three-member replica sets on separate hosts, with a failover test (kill a primary during a load).
- Running the Spark jobs on a real multi-node cluster (standalone, YARN or Kubernetes) and the dashboard on the sharded cluster.
- Spark MLlib versions of the sentiment and topic models.
- Kafka plus Spark Structured Streaming for real-time ingestion.
- Streaming ingestion with MongoDB change streams feeding incremental rollups.
- A job queue for ingestion and a shared cache (Redis) for multiple API workers.
- Authentication and role-based access for the API.
- Network analysis of retweets and mentions between accounts.
- Running the experiments on several laptops to show hardware sensitivity.

25 Conclusion

The project shows that a single 16 GB machine with MongoDB at its core can ingest, enrich, aggregate and serve analytics over almost three million social media posts. The decisive techniques were the ones the Big Data course emphasises: streaming instead of loading everything, bulk writes in batches, parallel processing, building indexes after bulk loads, indexes matched to query patterns, keeping computation inside the database, and pre-aggregating for interactive use. The Big Data Lab extension adds the distributed side: Spark processes the whole archive in about two and a half minutes, a partitioned Parquet lake answers ad-hoc questions up to 20× faster than CSV, and a sharded MongoDB cluster shows how the shard key decides which queries are targeted and which are broadcast. The experiments quantify each of these. The analytical layer works end to end, with clear limits: sentiment accuracy is modest and unvalidated on this domain, engagement is synthetic, and the dataset describes a specific influence operation rather than public opinion. Those limits are stated in the system itself as well as in this report.

26 References

1. Linvill, D. L., & Warren, P. L. (2018). *Troll Factories: The Internet Research Agency and State-Sponsored Agenda Building*. Working paper, Clemson University. Dataset published by FiveThirtyEight: https://github.com/fivethirtyeight/russian-troll-tweets (CC BY 4.0).
2. Barbieri, F., Camacho-Collados, J., Espinosa-Anke, L., & Neves, L. (2020). TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification. *Findings of EMNLP 2020*. https://github.com/cardiffnlp/tweeteval
3. Rosenthal, S., Farra, N., & Nakov, P. (2017). SemEval-2017 Task 4: Sentiment Analysis in Twitter. *Proceedings of SemEval-2017*. https://aclanthology.org/S17-2088/
4. Hutto, C. J., & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. *Proceedings of ICWSM 2014*.
5. Lee, D. D., & Seung, H. S. (1999). Learning the parts of objects by non-negative matrix factorization. *Nature*, 401, 788-791.
6. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *Proceedings of IEEE ICDM 2008*.
7. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
8. MongoDB documentation: Aggregation, Indexes (including the ESR guideline), Schema Validation, `$merge`, `$setWindowFields`. https://www.mongodb.com/docs/manual/
9. FastAPI documentation. https://fastapi.tiangolo.com/
10. React documentation. https://react.dev/
11. Apache Spark documentation: Spark SQL, DataFrames and Datasets Guide; Parquet Files. https://spark.apache.org/docs/latest/
12. Apache Parquet format specification. https://parquet.apache.org/docs/
13. MongoDB documentation: Sharding, Shard Keys, Hashed Sharding, Ranged Sharding. https://www.mongodb.com/docs/manual/sharding/
14. Brewer, E. (2012). CAP twelve years later: How the “rules” have changed. *Computer*, 45(2), 23-29.
15. Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM*, 51(1), 107-113.